AISE 4010

Deep Learning for Time Series Data

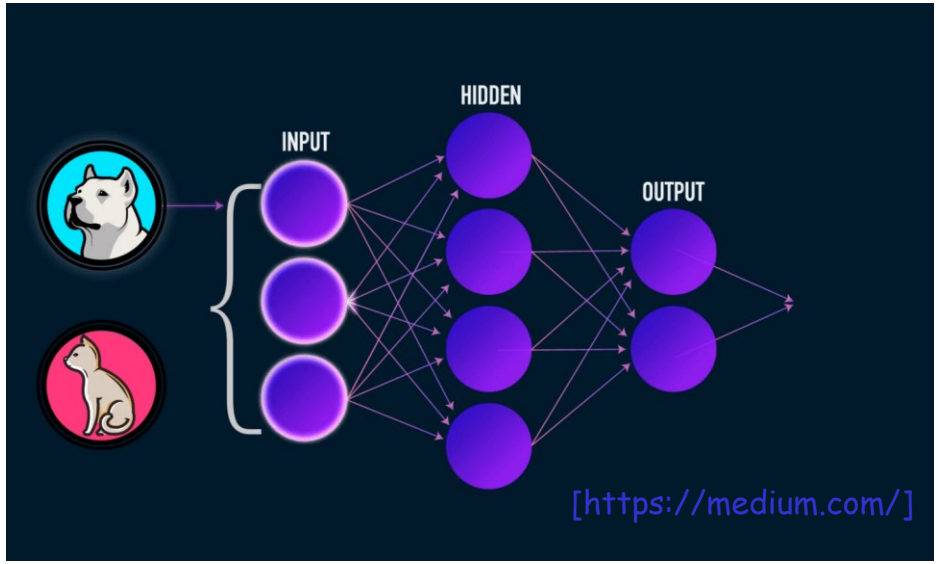
Lecture 3: Neural Networks for Time Series Predictions

September 23, 2026

3.1: Neural Network

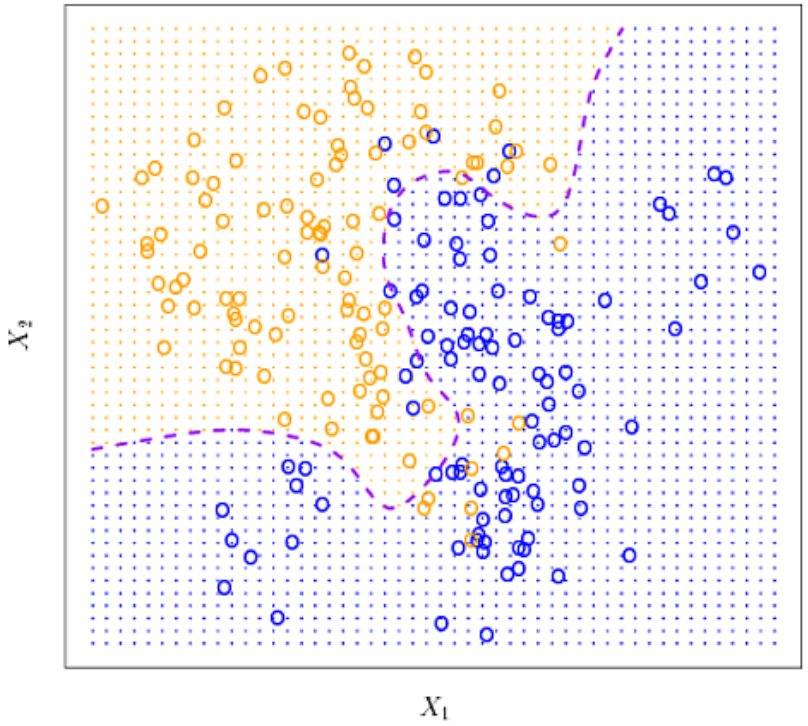
Neural Networks (NNs) for Time Series

- NNs offer a lot of promise for time series forecasting, such as the automatic learning of temporal dependence and the automatic handling of temporal structures like trends and seasonality.
- NNs approximate a mapping function from input variables to output variables
- NN capabilities:
  - Robust to Noise: NNs are robust to noise in input data and in the mapping function. They can even support learning and prediction in the presence of missing values.
  - Nonlinear: NNs do not make strong assumptions about the mapping function and readily learn linear and nonlinear relationships.
  - Multivariate Inputs: An arbitrary number of input features can be specified, providing direct support for multivariate forecasting.
  - Multi-step Forecasts: An arbitrary number of output values can be specified, providing direct support for multi-step and even multivariate forecasting.



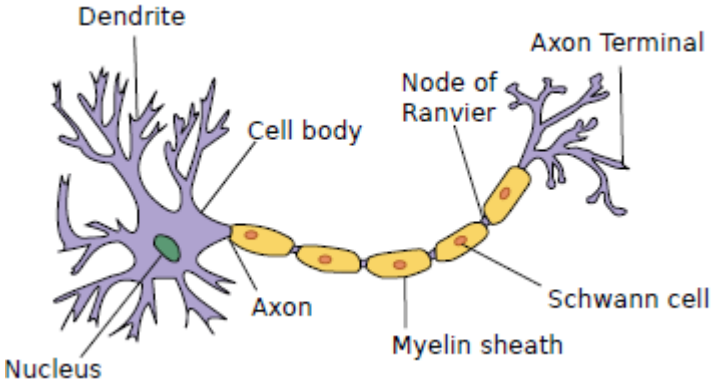
Linear Models Are Limited

- Linear regressors/classifiers (e.g., linear/logistic regression) model inputs based on linear combinations of features.
- In many cases, we need to deal with non-linearities
- This problem of nonlinearity has been studied for at least a century (e.g., decision trees with a sequence of binary decisions, kernel methods have been used for many decades to model nonlinear dependencies).
- Utilizing non-linear functions of input variables

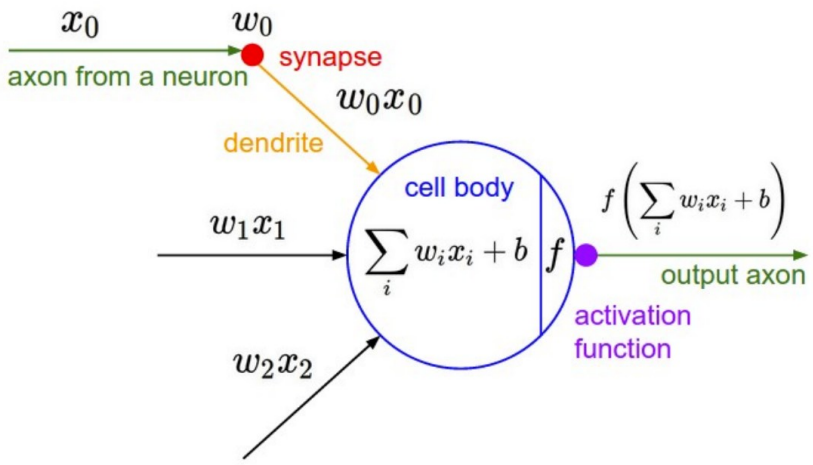


Neural Network: Biological Inspiration

- Non-linearity is something that the brain solves quite naturally.
- Neurons feed into other neurons which, in turn, feed into other neurons again. Consequently, we have a sequence of relatively simple transformations.
- Our brain has $\sim 10^{11}$ neurons, each of which communicates (is connected) to $\sim 10^{4}$ other neurons
- A biological neuron consists of dendrites (input terminals), the nucleus (CPU), the axon (output wire), and the axon terminals (output terminals), enabling connections to other neurons via synapses.
- Neural networks define functions of the inputs (features), computed by neurons.
- Artificial neurons are called units

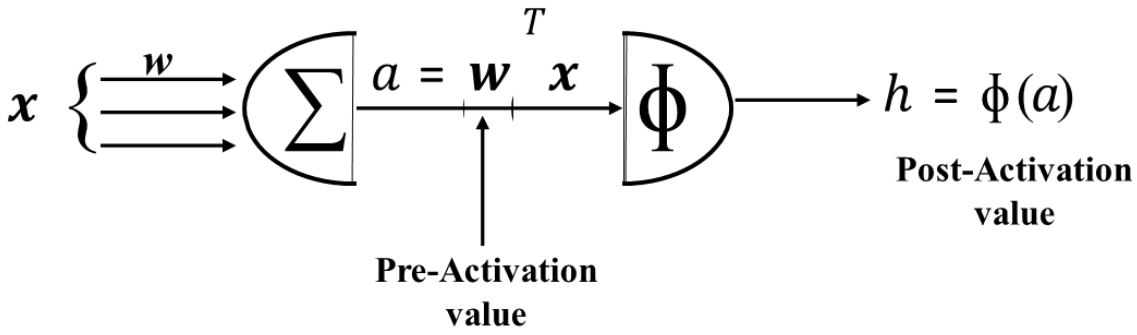


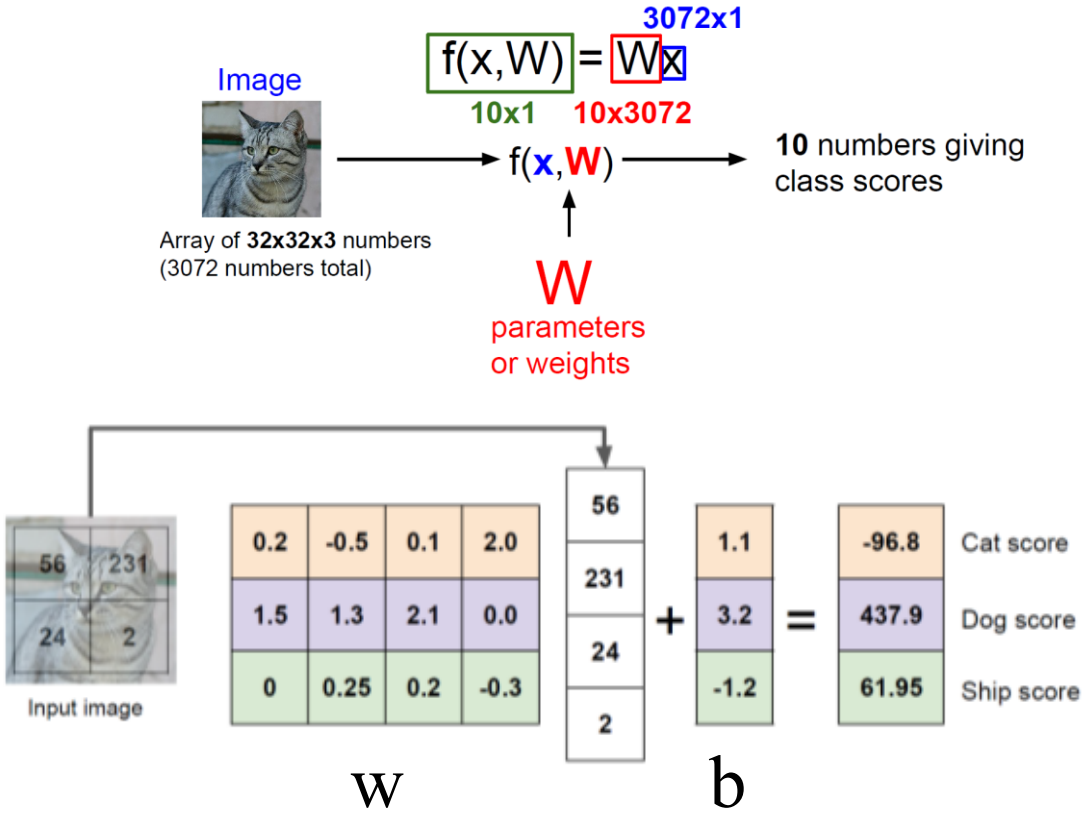
Real neuron (source: "Anatomy and Physiology"



Artificial neural network

Recap: Computational Unit



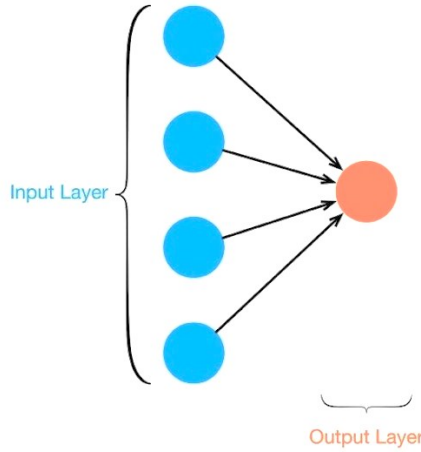


1. A simple linear combination of the inputs.

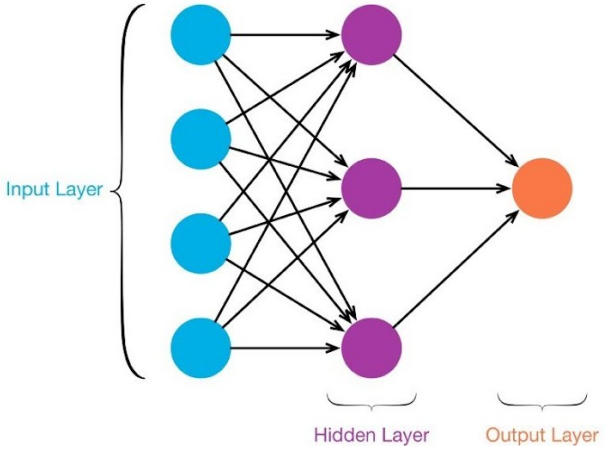
$$a_h = w_0 1 + w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + w_5 x_5 = w_0 + \boldsymbol{w}^T \boldsymbol{x}$$

2. The value computed by the 1st function is applied to a second function ϕ .
This function referred to as the activation function

Multi-layer Perceptron (MLP)



The simplest neural network is perceptron (a single input layer and an output node)



MLP contain multiple computational layers. The specific architecture of MLP is feed-forward neural network.

- We can overcome the limitations of linear models by incorporating one or more hidden layers. The easiest way to do this is to stack many fully connected layers on top of one another.
- MLP is a series of logistic regression models stacked on top of each other, with the final layer being either another logistic regression or a linear regression model (Classification/Regression, respectively)
- The capacity of the network increases with more hidden units and more hidden layers

Activation Functions

- Sigmoid

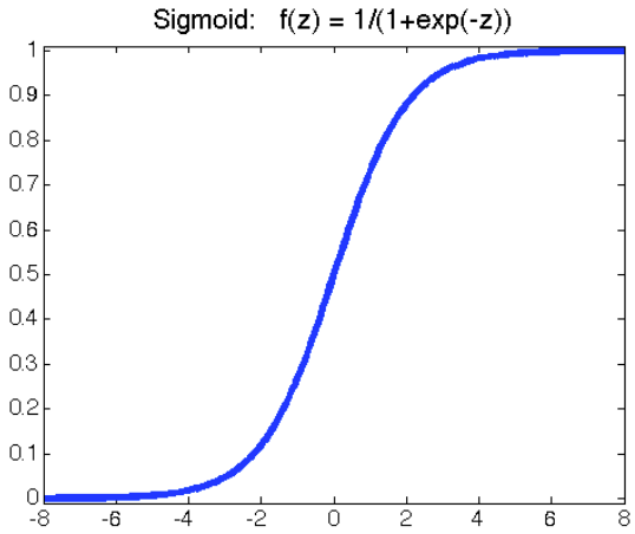
$$\sigma(z) = \frac{1}{1+\exp(-z)}$$

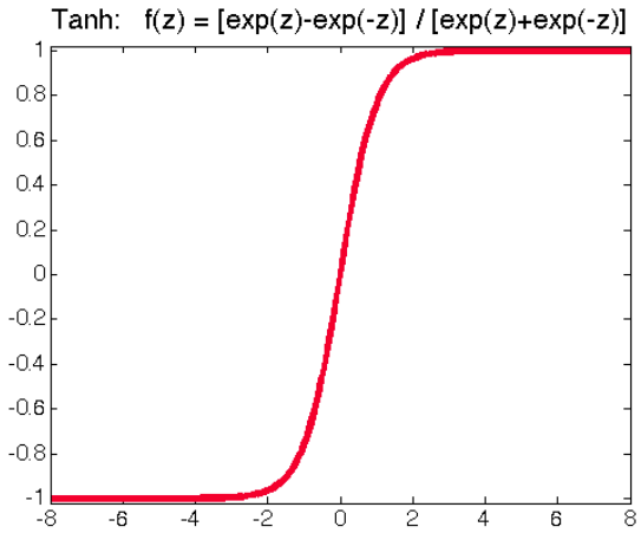
- Tanh

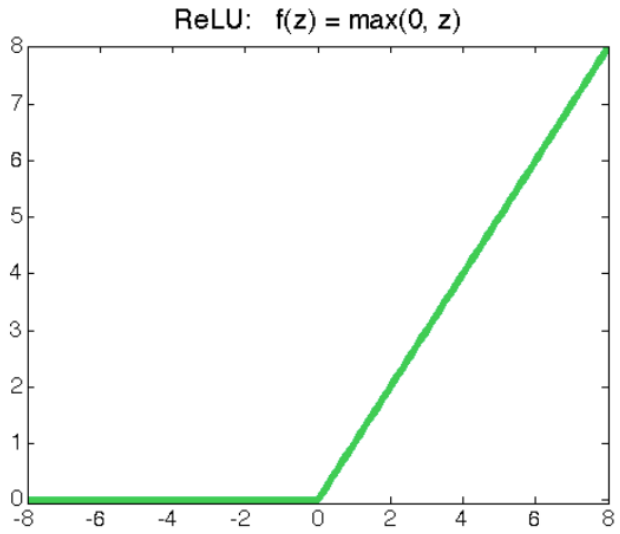
$$\tanh(z) = \frac{\exp(z)-\exp(-z)}{\exp(z)+\exp(-z)}$$

- ReLU (Rectified Linear Unit)

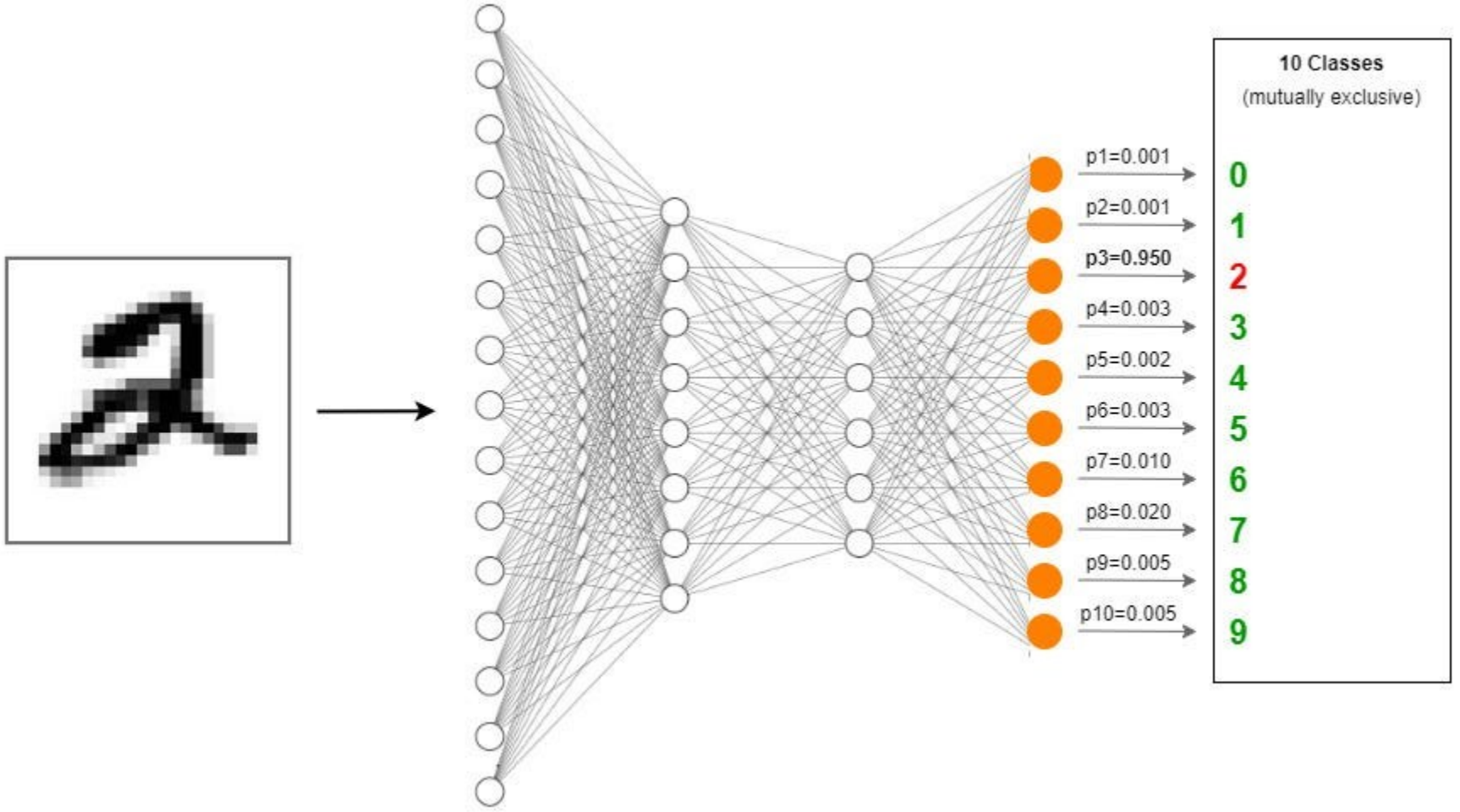
$$\mathrm{ReLU}(z) = \max(0, z)$$



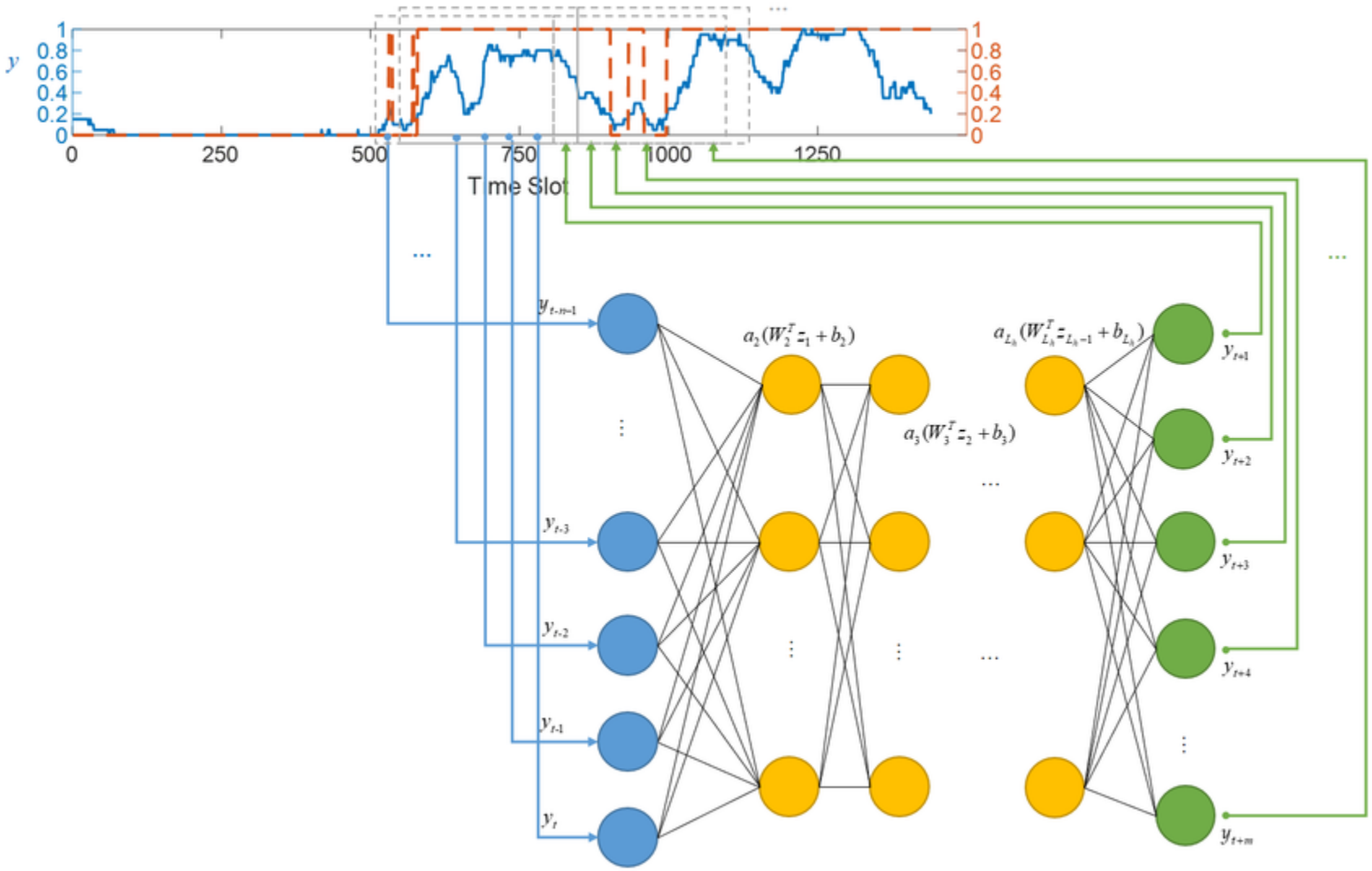




Example: Classifying Images From Pixels



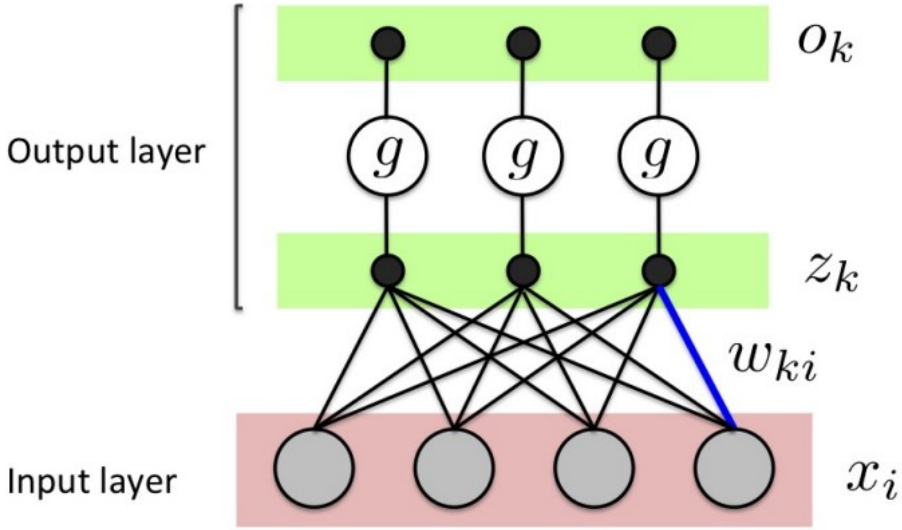
Example: Time Series Forecasting



3.2. How to Train Neural Networks?

Training NN

- We need to optimize a Loss/objective function over the whole training.
  - Cross Entropy loss for classification
  - Mean Squared Error loss for Regression
- The gradient of a composition function is computed using the backpropagation algorithm.
- The **backpropagation** is an efficient method for computing gradients needed to perform gradient-based optimization of the weights in a multi-layer network.



$$\frac{\partial E}{\partial w_{ki}} = \frac{\partial E}{\partial o_k}\frac{\partial o_k}{\partial z_k}\frac{\partial z_k}{\partial w_{ki}}$$

Gradient Descent

- Gradient Descent is an optimization method to find the minimum of loss function
- Update weights using backpropagation of error
- Start with random W and find a W that minimizes the loss
- Finds the direction of steepest descent (negative gradient (slope) of error)

Algorithm: Gradient Descent

Initialize weights: $W \sim \mathcal{N}(0, \sigma^2)$ (or zeros).

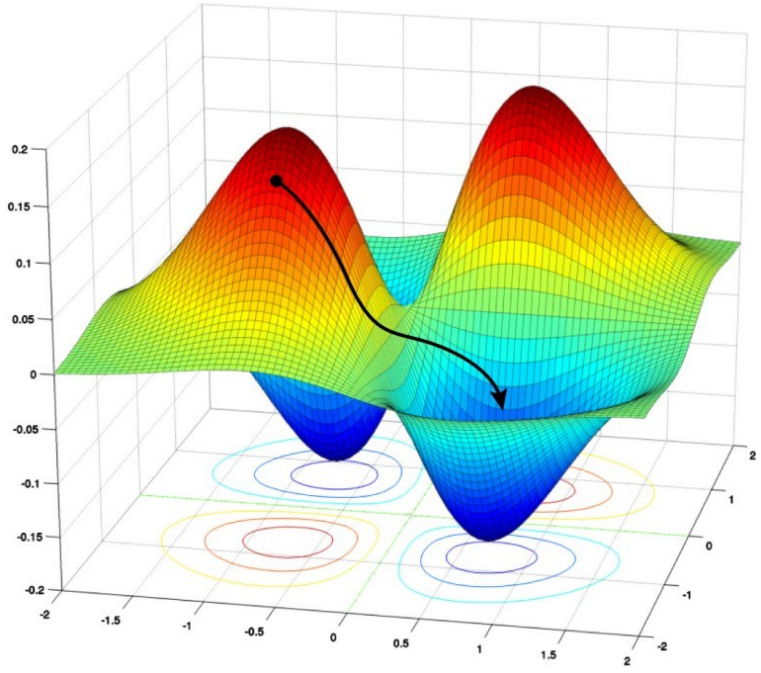
Repeat until convergence (or for $t = 1, \ldots, T$):

Compute gradient: $g \leftarrow \dfrac{\partial J(W)}{\partial W}$

Update weights: $W \leftarrow W - \eta\, g$ ($\eta > 0$ learning rate)

Return W

Convergence examples: stop if $\|g\|_2 \leq \varepsilon$, or $|J_t - J_{t-1}| \leq \varepsilon$, or after T iterations.

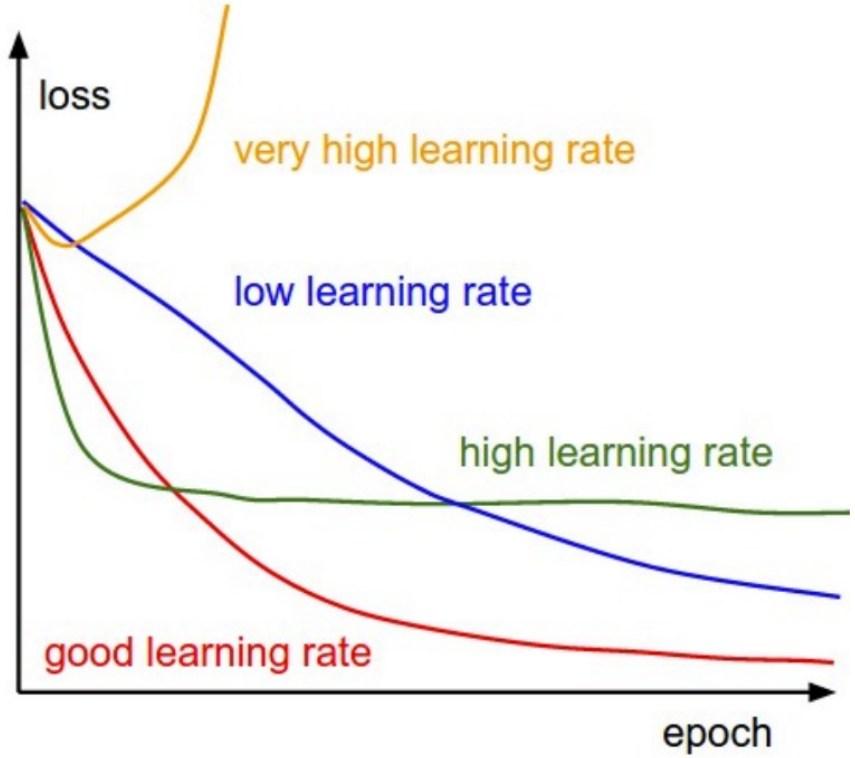


3.3. Difficulties in Training Neural Networks

Learning Rate

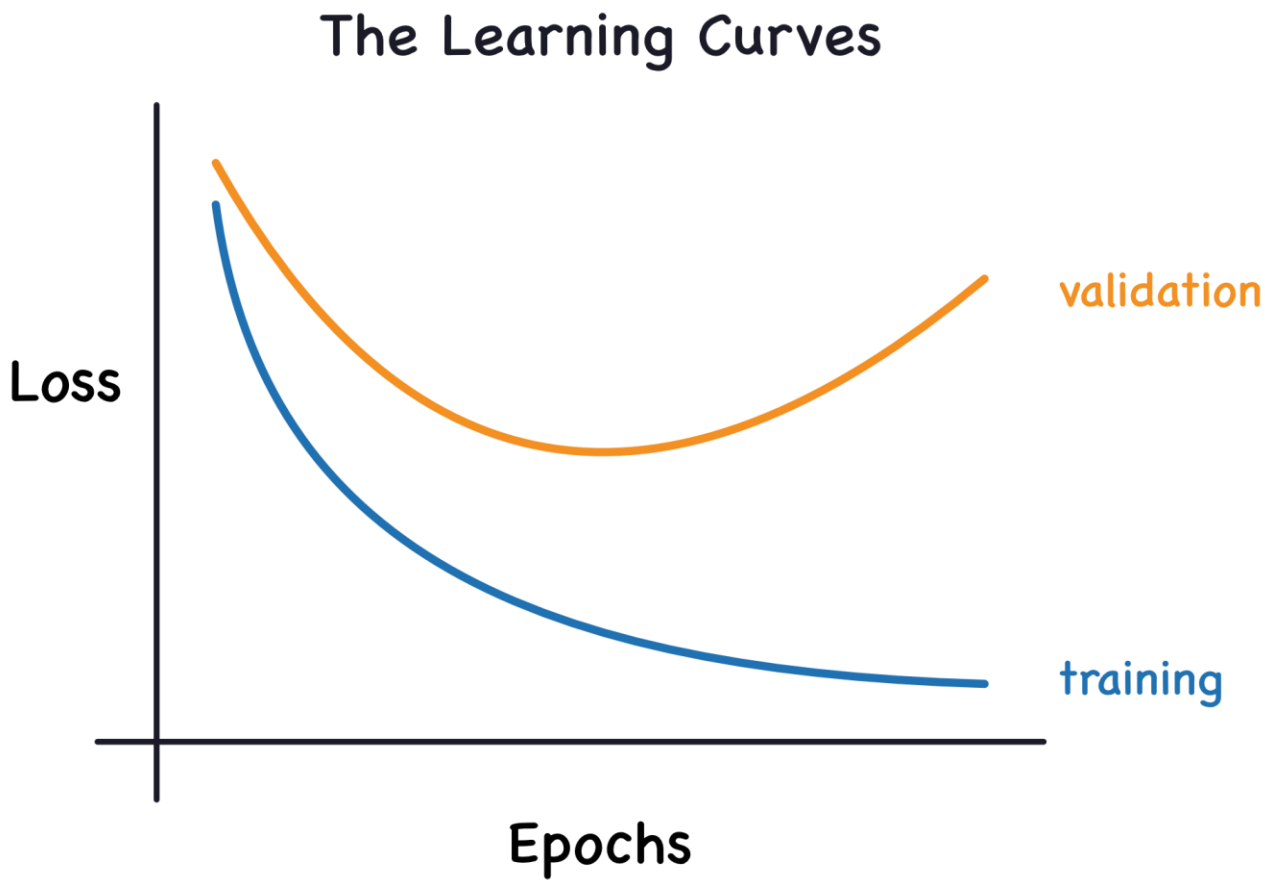
- large: overshoots, is unstable and diverges
- small: converges very slowly or get stuck in false local minima

- Solutions:
  - Learning rate schedules (step decay, exponential decay, cosine).
  - Adaptive optimizers (Adam, Adagrad).
  - Hyperparameter optimization (e.g., use grid search to tune)



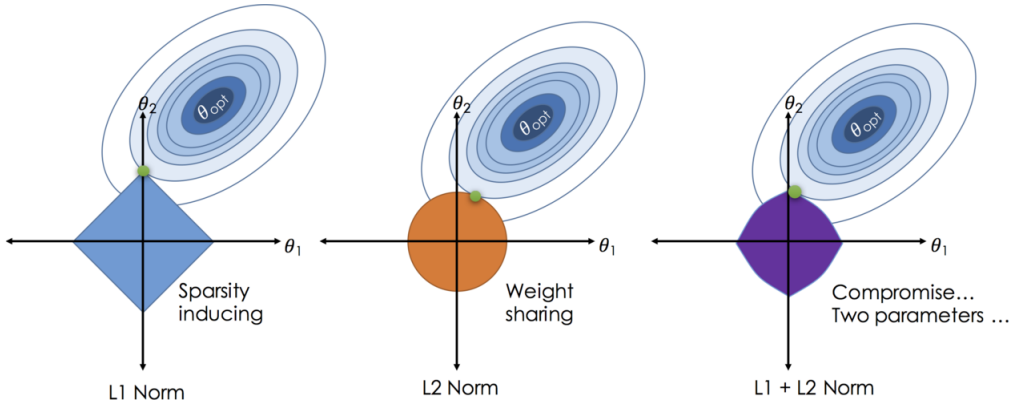
Overfitting

- Monitor loss during training (Check how your loss behaves during training)
- Model learns training data too well, fails to generalize → high train accuracy, low test accuracy

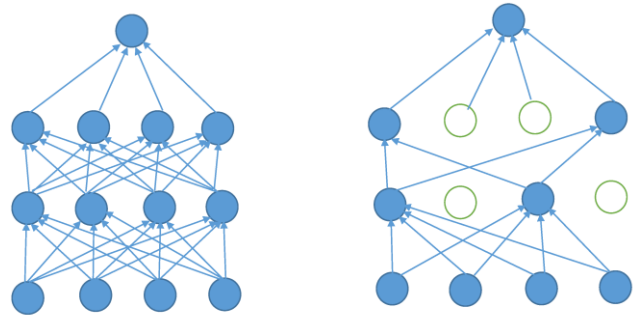


Overfitting Solutions

- Regularization – add term to loss
  - Ex: L_1or L_2 -regularization



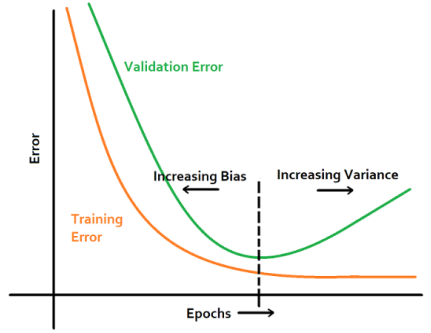
- Drop-out layers
  - In each forward pass, randomly set some neurons to zero



- Data augmentation (especially in vision, speech, text)
  - Random translation, rotations, distortion, crops, mixing etc. to enlarge dataset
  - data augmentation: use more training data or synthetic data (common in time series analysis).

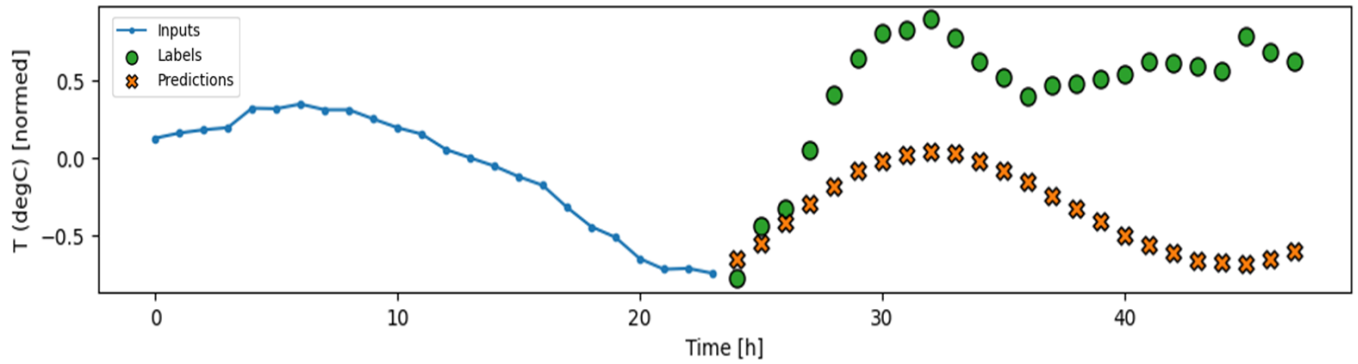


- Early stopping (stop before overfitting starts).



Underfitting

- Model is too simple or constrained → fails to capture patterns in data

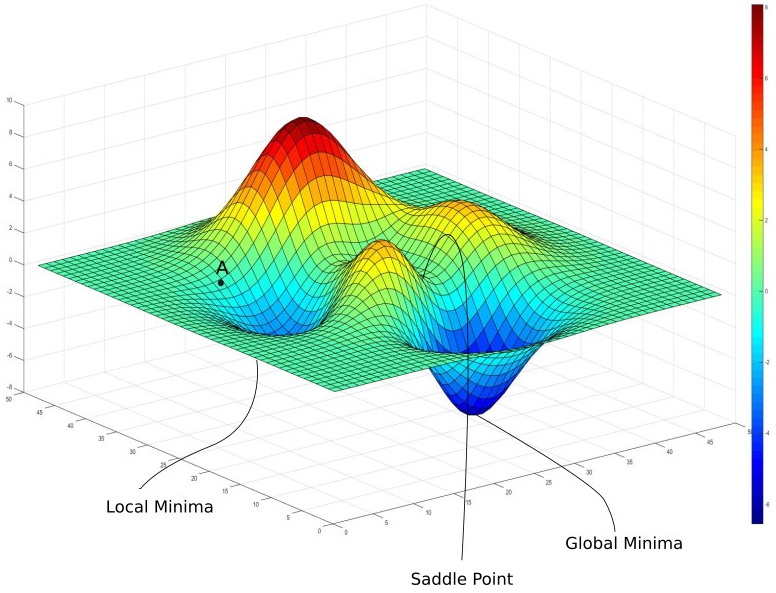


- Solutions:
  - Increase model capacity (more layers, units).
  - Reduce regularization strength.
  - Train longer/improve optimization.
  - Use better features or architectures.

Optimization Challenges

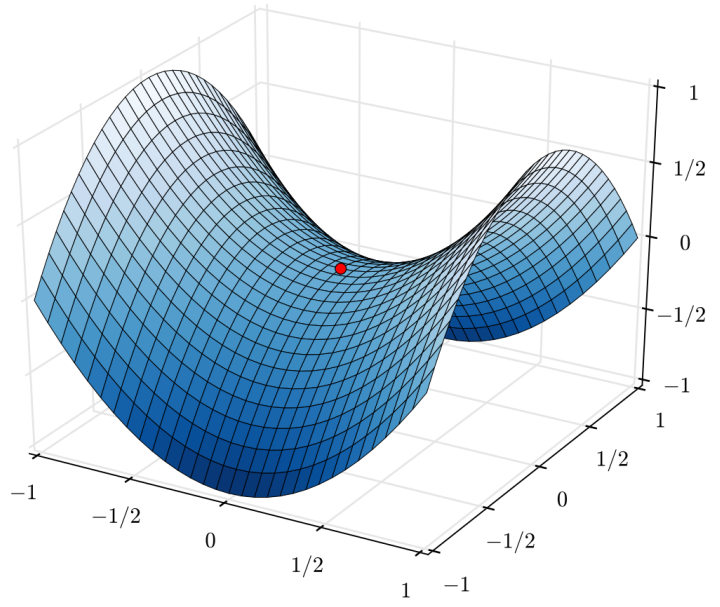
Loss can be difficult to optimize:

- Slow Convergence
- Local minimum: a minimum value that are not necessarily the lowest value of the entire function
- Saddle point: Unlike a minimum or maximum, the point is curved upwards in some directions and downwards in others
- Poor weight initialization



Solutions:

- Using stochastic gradient descent (SGD) rather than GD
- Advanced optimizers (SGD with momentum, Adam, RMSProp).
- Learning rate scheduling (decay, warm restarts, cosine annealing).
- Batch normalization / layer normalization (stabilize learning).
- Transfer learning for better weight initialization



Mini-Batch SGD

- In practice, datasets are huge (millions of samples).
- Running the full GD (Batch GD) is too expensive.
- Instead of computing the gradient using the entire dataset (Batch GD), we approximate it using just one random sample at a time.
- How about a compromise between full GD (entire training data) and SGD (one random training sample to update the model weights at each step): use a small random batch of size B
- Mini-batch SGD balances efficiency and performance: faster training (perfect for GPU), while still converging.
- Almost all modern NNs uses Mini-batch SGD + optimizers (Adam, RMSProp, etc.).

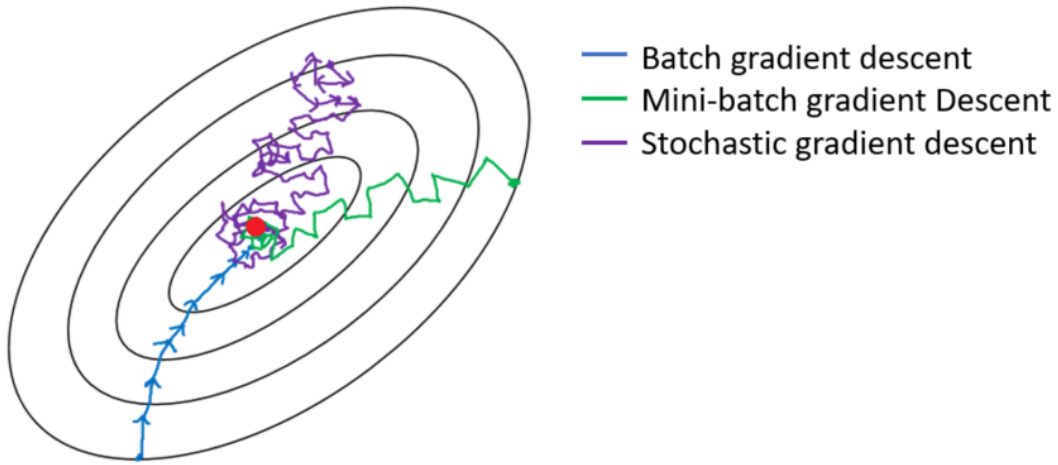
1. **Initialize weights:** $W \sim \mathcal{N}(0, \sigma^2)$
2. **Repeat until convergence (or max epochs):**
   a. Shuffle training data
   b. Partition data into mini-batches of size B
   c. For each mini-batch B_t:
      i. Compute gradient:

$$g \leftarrow \frac{1}{|B_t|} \sum_{i \in B_t} \frac{\partial \ell(W; x_i, y_i)}{\partial W}$$

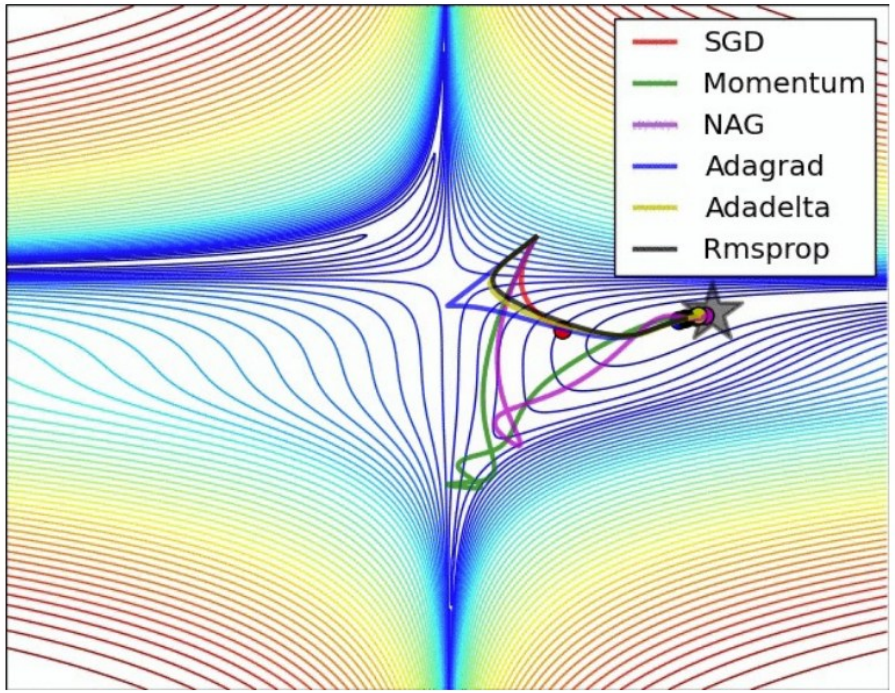
      ii. Update weights:

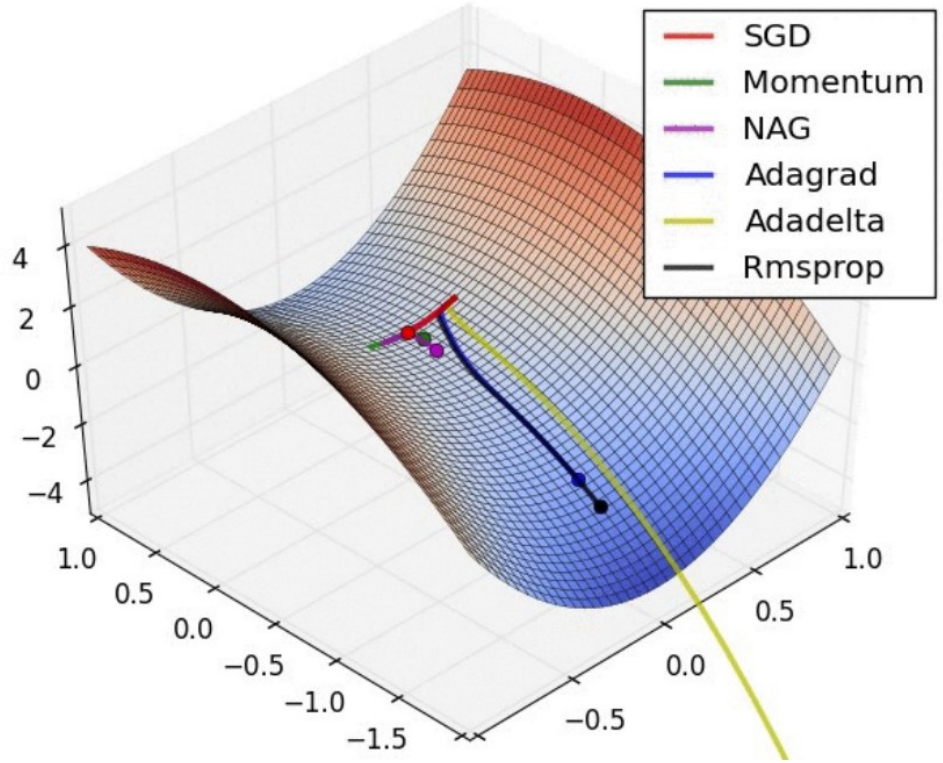
$$W \leftarrow W - \eta g$$

3. **Return** W



Comparing Optimization Methods





[cs231n]

3.4. MLP for Time Series Forecasting

Transform Time Series to a Supervised Learning Problem

- A time series is just a sequence: $(y_1, y_2, \ldots, y_T)$
- Time series forecasting can be framed as a supervised learning problem.
- Standard supervised learning algorithms expect input-output pairs (X, y) where you have input variables (X) and an output variable (y) and you use an algorithm to learn the mapping function from the input to the output: $y = f(X)$

Solution: Sliding Window/Lag Method

- Use past observations as features (inputs).
- Use the next observation (or multiple) as the target (output).

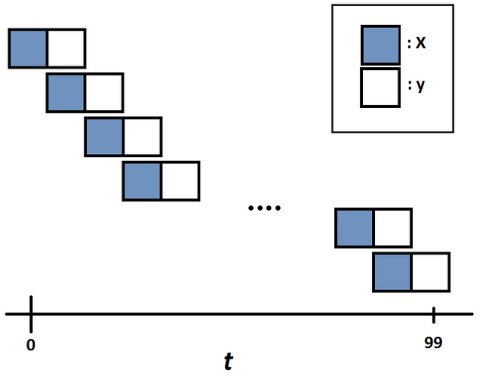
$$X_t = [y_{t-1}, y_{t-2}, \ldots, y_{t-p}], \quad y_t = \text{next value}$$



- Why sliding window is important:
  - The order between the observations is preserved
  - Any standard linear/nonlinear regression/classification model may be applied (e.g., MLP, SVM, LR, etc.)
  - Works for univariate (one series), multivariate (multiple variables per time step), and multi-step forecasting (predicting several future values).

Sliding Window – Univariate

- For example, a dataset of 100 rows (x 1 column), we want to use the previous input(t-1) to determine t.
- Data can be sliced from the shape (100, 1) to X (99, 1, 1) and y (99, 1, 1). Matrix (of 100row x 1column) is transformed into 2 tensors: 99 matrices each of 1 row and 1 column where the row is the number of time and column is the number of features.
- We lose one row because we can not include t=0 into y as I have missing X values at t = -1.
- The input values can be increased to any length(as long as not greater than the dataset's size).



Example:

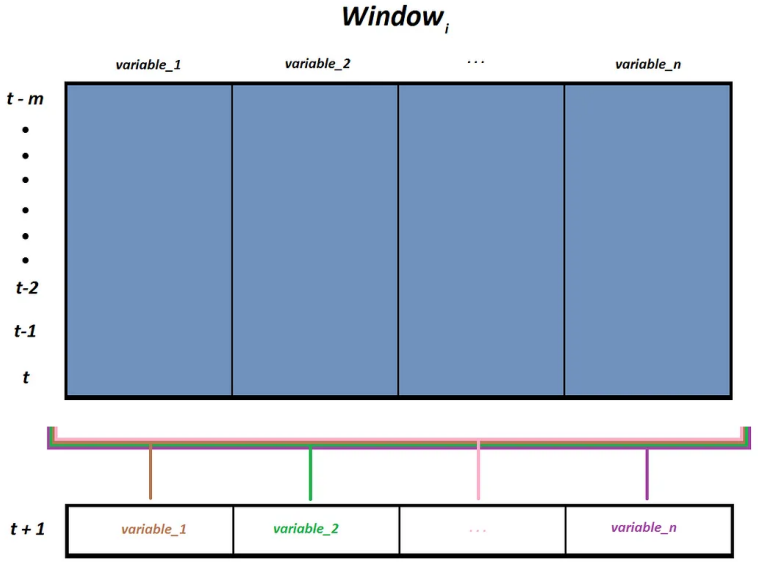
A series $= [10, 20, 30, 40, 50]$ with window size $= 3$

$$X = [10, 20, 30] \rightarrow y = 40$$
$$X = [20, 30, 40] \rightarrow y = 50$$

Sliding Window – Multivariate, Multi Steps

With Multiple Variates

- Multivariate Time Series, where two or more variables are observed at each time (such as temperature, pressure and humidity at each hour).
- Multivariate data is often more difficult to work with. Often many of classical methods do not perform well.
- Multivariate time series analysis considers simultaneously multiple time series.



With Multiple Steps

- The number of time steps ahead to be forecasted is important.
  - One-step Forecast: This is where the next time step (t+1) is predicted.
  - Multi-step Forecast: This is where two or more future time steps (t+1, t+2, …, t+n) are to be predicted .

MLP for Time Series

- MLPs can be applied to time series forecasting.
- A challenge with using MLPs for time series forecasting is in the preparation of the data.
- Lag observations must be flattened into feature vectors.

  - Univariate MLP Models
  - Multivariate MLP Models
  - Multi-Step MLP Models
  - Multivariate Multi-Step MLP Models

Data Splitting (Train/Validation/Test Splits)

- Choose a window size p:
  - Example: $p = 3 \rightarrow$ use the last 3 values to predict the next one.
- Convert time series into supervised samples:

  Series: $[10, 20, 30, 40, 50, \ldots]$

  $X = [10,20,30] \rightarrow y = 40$

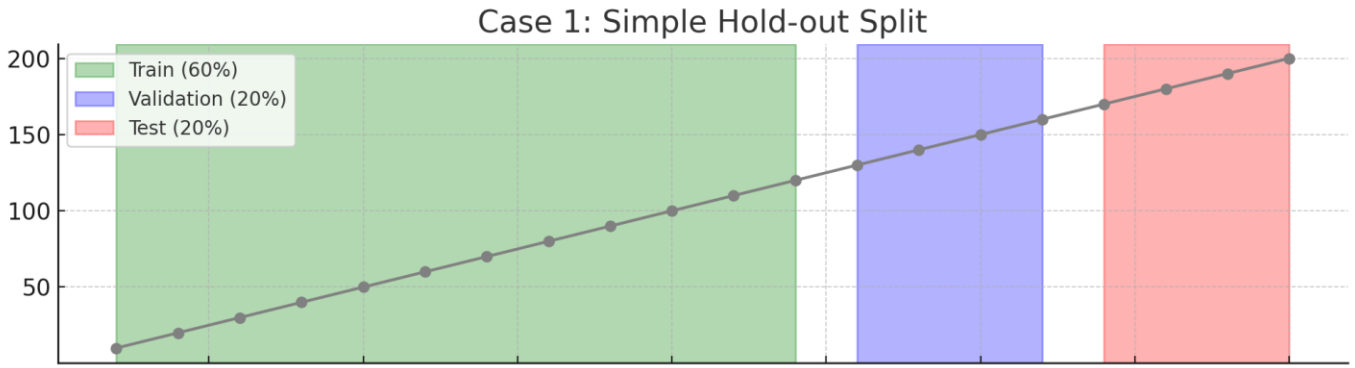
  $X = [20,30,40] \rightarrow y = 50$

  $X = [30,40,50] \rightarrow y = 60$

  $X = [40,50,60] \rightarrow y = 70$

  $X = [50,60,70] \rightarrow y = 80$

  …



  Our dataset looks like:
  - $X = [[10,20,30], [20, 30, 40], [30,40,50], [40,50,60], [50,60,70], \ldots]$
  - $y = [40, 50,60,70,80, \ldots]$

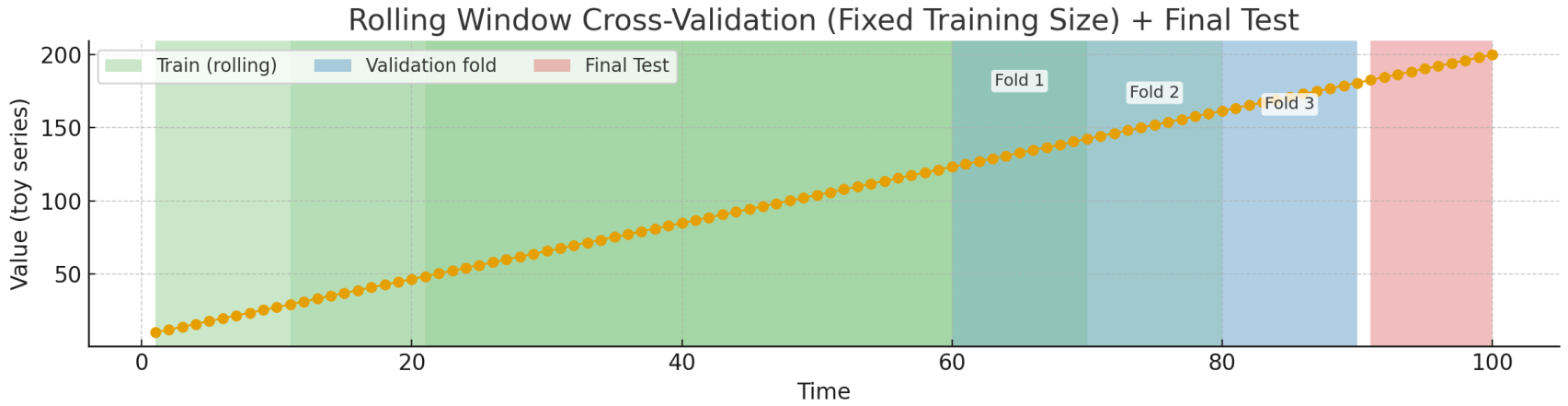
- Simple Hold-out Split (most common)
  - Split the time series with no shuffling.
  - Example (first 60% for training, next 20% for validation, last 20% for test)

Cross Validation (Rolling Window)

- Split the time series into training and test sets (Ex. first 90% for training, next 10% for test)
- On the training set, create multiple folds while preserving temporal order (Rolling: training set slides/rolls forward with fixed size).
- More common when non-stationarity or concept drift is suspected e.g., stock market, user behavior, network traffic (recent data is more important).
- Training set has a fixed size (window). As you move forward, the window slides: you drop old points and add new ones. The validation set is the next block.

  Example: a series with 100 data points $y1, y2, \ldots, y100$ where $y91, \ldots, y100$ is separated for test. Fixed train window is 60 points and validation horizon is 10 points.

  - $Fold\ 1: Train = [y1, \ldots, y60\ ], validate = [y61, \ldots, y70]$
  - $Fold\ 2: Train = [y11, \ldots, y70\ ], validate = [y71, \ldots, y80]$
  - $Fold\ 3: Train = [y21, \ldots, y80\ ], validate = [y81, \ldots, y90]$

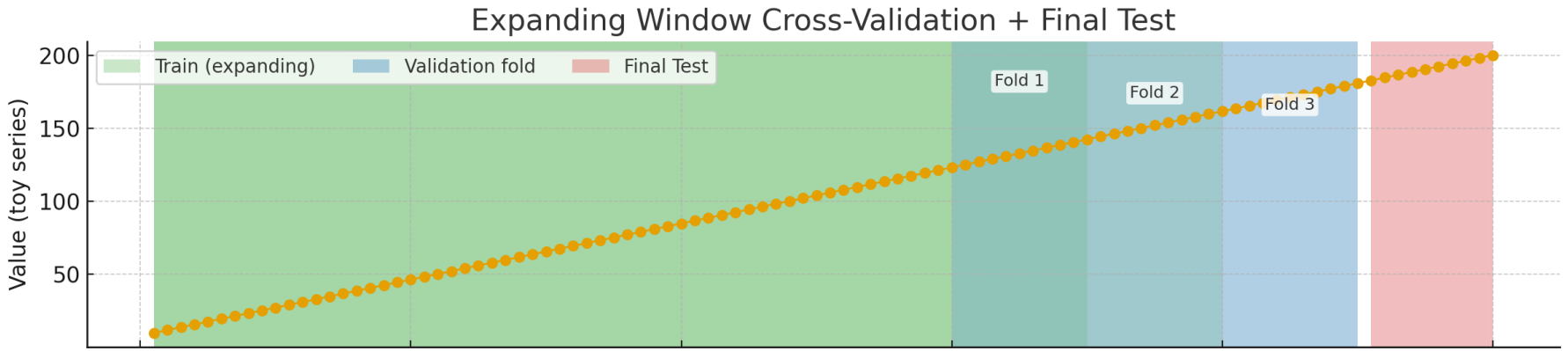


Cross Validation (Expanding Window)

- Split the time series into training and test sets (Ex. first 80% for training, next 20% for test)
- On the training set, create multiple folds while preserving temporal order (Expanding: training set grows).
- Most common in forecasting because we always have all past data up to today and predict the future (e.g., predicting energy demand, sales, weather).
- Training set starts small and expands forward. The validation set is always the next block in time.
- You never drop old data; model keeps memory of the full history.

  Example: a series with 100 data points $y1, y2, \ldots, y100$ where $y91, \ldots, y100$ is separated for test. Validation horizon is 10 points.

  - $Fold\ 1: Train = [y1, \ldots, y60\ ], validate = [y61, \ldots, y70]$
  - $Fold\ 2: Train = [y1, \ldots, y70\ ], validate = [y71, \ldots, y80]$
  - $Fold\ 3: Train = [y1, \ldots, y80\ ], validate = [y81, \ldots, y90]$



Univariate MLP Models

- MLPs can be used to model univariate time series forecasting problems.

- Univariate time series are datasets comprised of a single series of observations with a temporal ordering, and a model is required to learn from the series of past observations to predict the next value in the sequence.

- MLP model will learn a function that maps a sequence of past observations as input to an output observation.
- The sequence of observations must be transformed into multiple examples from which the model can learn.

Univariate MLP Models - Prepare Data

- Restructure the data to look like a supervised learning problem.
- Divide the sequence into multiple input/output patterns called samples
- Use previous time steps as input variables and use the next time step as the output variable (**siding window)**

Example: 3 time steps are used as input and 1 time step is used as output for the 1-step prediction that is being learned (window size = 3 time steps)

```
[10, 20, 30, 40, 50, 60, 70, 80, 90]
```

```
X, y
10, 20, 30 40
20, 30, 40 50
30, 40, 50 60
...
```

Univariate MLP Models - MLP Model

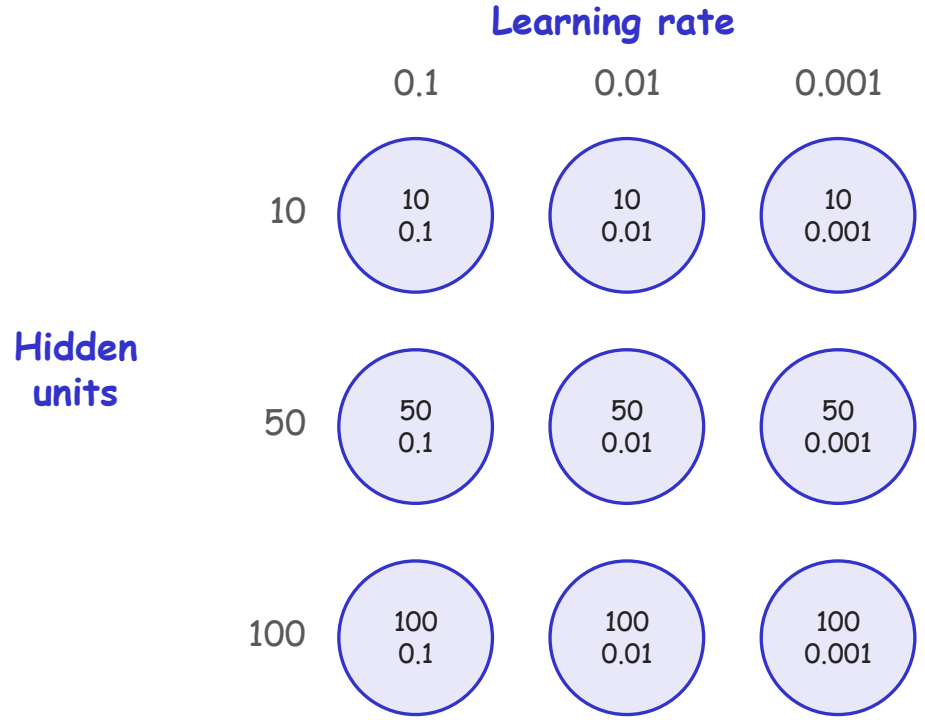
- Decision about Input, hidden layer and output size
- Optimization (ADAM, SGD, …)
- Loss function (MSE)
- Number of iterations/epochs
- Fitting model
- Prediction
- Evaluation

```
# define model
model = Sequential()
model.add(Dense(100, activation='relu', input_dim=n_steps))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
# fit model
model.fit(X, y, epochs=2000, verbose=0)
# demonstrate prediction
x_input = array([70, 80, 90])
x_input = x_input.reshape((1, n_steps))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

prediction: `[[101.10744]]`

Hyperparameter Optimization (HPO): Grid Search

- Weights are learned by gradient descent. **Hyperparameters** are fixed before training (e.g., hidden units, learning rate, window size, epochs, batch size, activation, etc.)
- **Grid search**: simplest HPO approach, lists candidate values for each hyperparameter, train one model per combination, keep the one with the lowest **validation** loss (e.g., MSE, RMSE).
- For time series, score every combination with expanding-window CV (no shuffling). Use the test set only once, at the end.



3 × 3 = **9 models** to compare; × 3 CV folds = **27 training runs**

1 Define the grid of candidate values → 2 Train + validate every combo on each CV fold → 3 Average validation MSE, pick the lowest → 4 Retrain best combo on all training data, test once

Grid Search Tuning: Try Every Combination

Example grid for HPO of the univariate MLP forecasting problem:

Hidden units	Learning rate	Batch size
{16, 32, 64}	{0.01, 0.001}	{16, 32}

Cost

$3 \times 2 \times 2 = 12$ MLPs

Illustrative validation RMSE ↓ lower is better

Batch size = 16

units \ learning rate →	0.01	0.001
16	4.8	3.2
32	3.7	**2.1**
64	4.1	2.4

Batch size = 32

units \ learning rate →	0.01	0.001
16	5.0	3.5
32	3.9	2.3
64	4.4	2.6

Best validation result: 32 hidden units • learning rate = 0.001 • batch size = 16 → RMSE = 2.1

Grid Search for MLP: Code Example

```python
from itertools import product
import numpy as np
import tensorflow as tf
from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import TimeSeriesSplit

# X_train and y_train contain TRAINING DATA ONLY
X_train = np.asarray(X_train)
y_train = np.asarray(y_train)
n_steps = X_train.shape[1]

# Hyperparameter grid
grid = {'units': [16, 32, 64],'lr': [0.01, 0.001]}

# Expanding-window time-series CV
tscv = TimeSeriesSplit(n_splits=3)
results = {}

for units, lr in product(grid['units'], grid['lr']):

    fold_mse = []

    for tr, val in tscv.split(X_train):
        tf.keras.backend.clear_session()
        # Build a new model for this fold
        model = Sequential([
            Input(shape=(n_steps,)),
            Dense(units, activation='relu'),
            Dense(1)])
        model.compile(
            optimizer=Adam(learning_rate=lr), loss='mse')
        model.fit(
            X_train[tr], y_train[tr], epochs=200, verbose=0)
        mse = model.evaluate(
            X_train[val], y_train[val], verbose=0)

        fold_mse.append(mse)
    # Average validation error across folds
    results[(units, lr)] = np.mean(fold_mse)

best = min(results, key=results.get)
print("Best configuration (units, lr):", best)
print("Best CV MSE:", results[best])
```

Cost grows multiplicatively

3 units × 3 lr × 3 folds will need 27 runs

+ window {3, 6, 12} + batch {16, 32} → 162 runs

- Search the learning rate on a log scale (0.1, 0.01, 0.001), then refine around the best value.
- Beyond 2 to 3 hyperparameters, switch to random search or Bayesian optimization (KerasTuner, Optuna). Future Lectures
- Never pick hyperparameters using the test set.

```
Best configuration (units, lr): (64, 0.001)
Best CV MSE: 0.013576709665358067
```

Multivariate MLP Models

- Multivariate time series data needs a little bit more preparation
- Multivariate time series data means data where there is more than one observation for each time step.

- There are two main models that we may require with multivariate time series data; they are:

  - Multiple Input Series
  - Multiple Parallel Series.

Multivariate MLP Models - Multiple Input Series

- A problem may have two or more parallel input time series and an output time series that is dependent on the input time series.
- The input time series are parallel because each series has an observation at the same time step.

```
# define input sequence
in_seq1 = array([10, 20, 30, 40, 50, 60, 70, 80, 90])
in_seq2 = array([15, 25, 35, 45, 55, 65, 75, 85, 95])
out_seq = array([in_seq1[i]+in_seq2[i] for i in range(len(in_seq1))])
# convert to [rows, columns] structure
in_seq1 = in_seq1.reshape((len(in_seq1), 1))
in_seq2 = in_seq2.reshape((len(in_seq2), 1))
out_seq = out_seq.reshape((len(out_seq), 1))
# horizontally stack columns
dataset = hstack((in_seq1, in_seq2, out_seq))
```

```
Dataset: [[ 10  15  25]
          [ 20  25  45]
          [ 30  35  65]
          [ 40  45  85]
          [ 50  55 105]
          [ 60  65 125]
          [ 70  75 145]
          [ 80  85 165]
          [ 90  95 185]]
```

```
Output Sequence: [[ 25  45  65  85 105 125 145 165 185]]
```

Multivariate MLP Models - Prepare Data

- We must structure these data into samples with input and output samples.
- We need to split the data into samples maintaining the order of observations across the two input sequences.

Example: If we choose three input time steps:

- X component has a three-dimensional structure.
- We have 2 parallel time series, 7 samples and 3 time steps per sample.

```
(7, 3, 2) (7,)
[[10 15]
 [20 25]
 [30 35]] 65
[[20 25]
 [30 35]
 [40 45]] 85
[[30 35]
 [40 45]
 [50 55]] 105
[[40 45]
 [50 55]
 [60 65]] 125
[[50 55]
 [60 65]
 [70 75]] 145
[[60 65]
 [70 75]
 [80 85]] 165
[[70 75]
 [80 85]
 [90 95]] 185
```

Multivariate MLP Models - MLP Model

- We must flatten the temporal structure of the input samples (a vector as the input).
- In this case (multivariate input), we will have multiple vectors (one for each time step).

```
[[10 15]
[20 25]
[30 35]]
```

→ `[10, 15, 20, 25, 30, 35]`

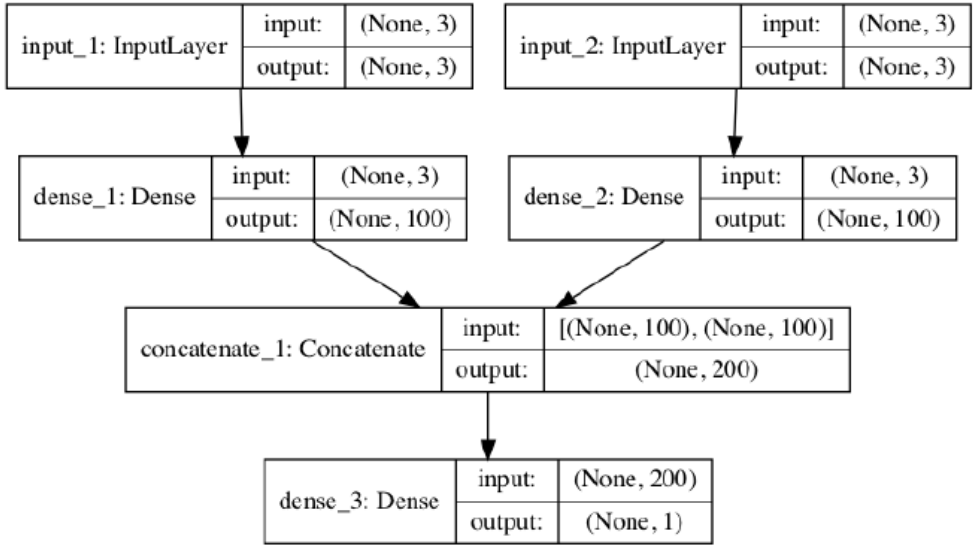
```
# flatten input
n_input = X.shape[1] * X.shape[2]
X = X.reshape((X.shape[0], n_input))
# define model
model = Sequential()
model.add(Dense(100, activation='relu', input_dim=n_input))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
# fit model
model.fit(X, y, epochs=2000, verbose=0)
# demonstrate prediction
x_input = array([[80, 85], [90, 95], [100, 105]])
x_input = x_input.reshape((1, n_input))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

prediction: `[[207.02144]]`

Multi-headed MLP Model

- A more elaborate way to model the problem is:
  - handling each input series by a separate MLP.
  - the output of each of these submodels can be combined before a prediction is made for the output sequence.

- It may offer more flexibility or better performance!




```python
# separate input data
X1 = X[:, :, 0]
X2 = X[:, :, 1]
# first input model
visible1 = Input(shape=(n_steps,))
dense1 = Dense(100, activation='relu')(visible1)
# second input model
visible2 = Input(shape=(n_steps,))
dense2 = Dense(100, activation='relu')(visible2)
# merge input models
merge = concatenate([dense1, dense2])
output = Dense(1)(merge)
model = Model(inputs=[visible1, visible2], outputs=output)
model.compile(optimizer='adam', loss='mse')
# fit model
model.fit([X1, X2], y, epochs=2000, verbose=0)
# demonstrate prediction
x_input = array([[80, 85], [90, 95], [100, 105]])
x1 = x_input[:, 0].reshape((1, n_steps))
x2 = x_input[:, 1].reshape((1, n_steps))
yhat = model.predict([x1, x2], verbose=0)
print(yhat)
```

prediction: `[[206.29399]]`

Multiple Parallel Series (Multivariate Forecasting)

- An alternate time series problem is the case where there are multiple parallel time series and a value must be predicted for each time series for the next time step.
- Again, the data must be split into input/output samples in order to train a model.

Input:

```
10, 15, 25
20, 25, 45
30, 35, 65
```

Output:

```
40, 45, 85
```

- X is three-dimensional, including the number of samples (6), the number of time steps chosen per sample (3), and the number of parallel time series or features (3).
- y is two-dimensional as we might expect for the number of samples (6) and the number of time variables per sample to be predicted (3).

```
(6, 3, 3) (6, 3)
[[10 15 25]
 [20 25 45]
 [30 35 65]] [40 45 85]
[[20 25 45]
 [30 35 65]
 [40 45 85]] [ 50  55 105]
[[ 30  35  65]
 [ 40  45  85]
 [ 50  55 105]] [ 60  65 125]
[[ 40  45  85]
 [ 50  55 105]
 [ 60  65 125]] [ 70  75 145]
[[ 50  55 105]
 [ 60  65 125]
 [ 70  75 145]] [ 80  85 165]
[[ 60  65 125]
 [ 70  75 145]
 [ 80  85 165]] [ 90  95 185]
```

Multiple Parallel Series - Prepare Data

- We must flatten the 3D structure of the input data samples to a 2D structure of [samples, features], where lag observations are treated as features by the model.

```
# flatten input
n_input = X.shape[1] * X.shape[2]
X = X.reshape((X.shape[0], n_input))
```

- The model output will be a vector, with one element for each of the different time series.

```
# determine the number of outputs
n_output = y.shape[1]
```

Multiple Parallel Series – Vector Output MLP Model

- With this model we can predict the next value in each of the three parallel series by providing an input of three time steps for each series.
- The shape of the input for making a single prediction must be 1 sample, 3 time steps and 3 features, or [1, 3, 3]. We flatten this to [1, 6] to meet the expectations of the model.

prediction: `[[100.35203  104.948586 205.59125 ]]`

```
# flatten input
n_input = X.shape[1] * X.shape[2]
X = X.reshape((X.shape[0], n_input))
n_output = y.shape[1]
# define model
model = Sequential()
model.add(Dense(100, activation='relu', input_dim=n_input))
model.add(Dense(n_output))
model.compile(optimizer='adam', loss='mse')
# fit model
model.fit(X, y, epochs=2000, verbose=0)
# demonstrate prediction
x_input = array([[70,75,145], [80,85,165], [90,95,185]])
x_input = x_input.reshape((1, n_input))
yhat = model.predict(x_input, verbose=0)
print(yhat)
```

- Note: With multiple input series, another way to model the problem is Multi-output MLP. Each output series can be handled by a separate output MLP model. We can refer to this as a multi-output MLP model. It may offer er more flexibility or better performance.

Other Categories of MLP Forecasting

- Multi-step MLP Models:
  - In practice, there is little difference to the MLP model in predicting a vector output that represents different output variables (as in the previous example) or a vector output that represents multiple time steps of one variable.
  - The MLP can output a vector directly that can be interpreted as a multi-step forecast.

- Multivariate Multi-step MLP Models
  - It is possible to mix and match the different types of MLP models presented for a different problem.
  - This applies to time series forecasting problems that involve multivariate and multi-step forecasting, but it may be a little more challenging, particularly in preparing the data and defining the shape of inputs and outputs for the model.